

Ham Potansiyelden Uzman Zekaya: Modern Bir Büyük Dil Modelinin Yaşam Döngüsü

Foundation Modellerden RAG Destekli Uygulamalara, Teknolojinin Perde Arkası



Foundation Model



Fine-Tuning & PEFT



RAG ile Temellendirilmiş Uygulama

Her Şeyin Başlangıcı: Temel Model (Foundation Model)

Tanım:

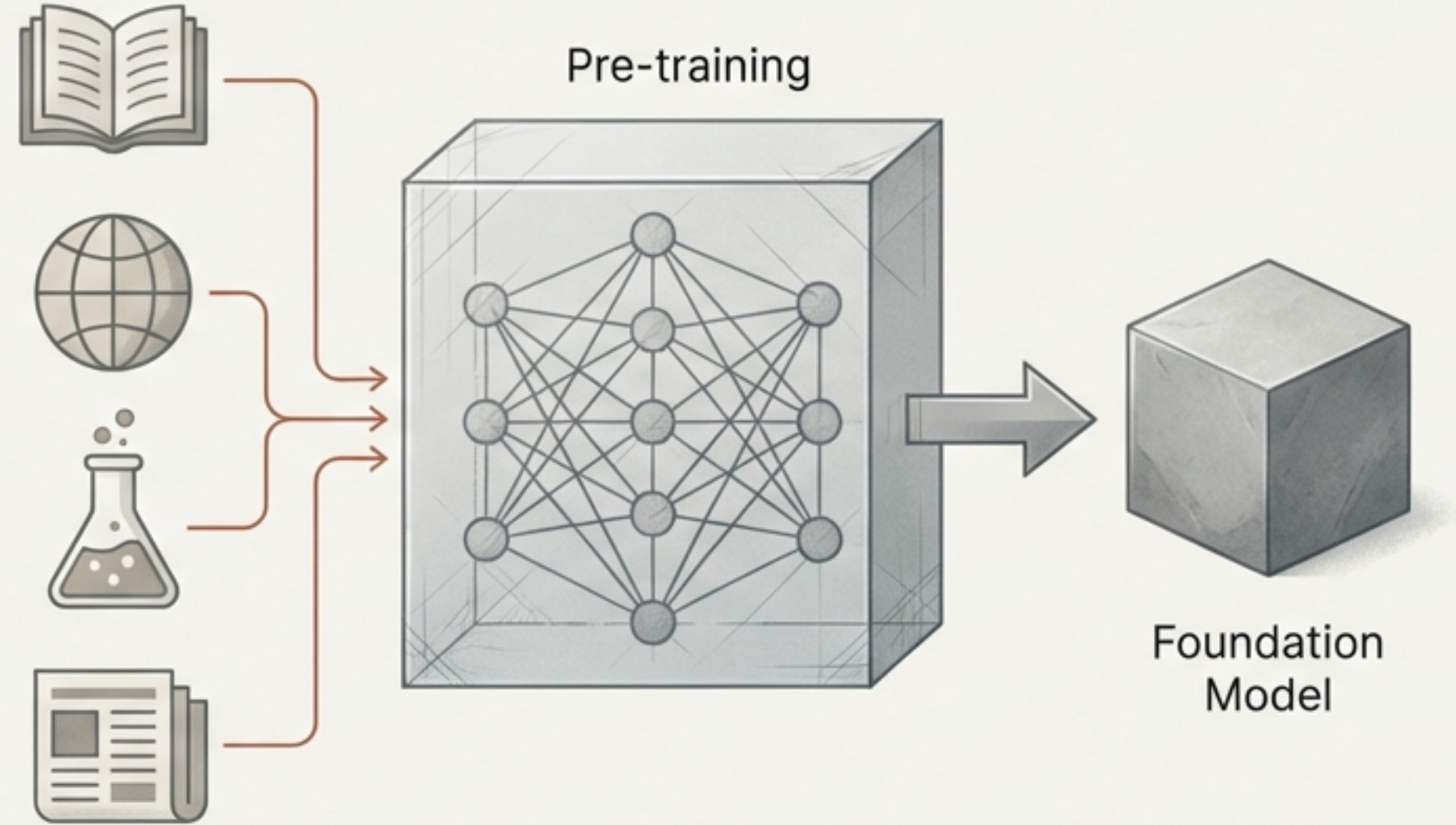
İnternet, kitaplar ve makaleler gibi devasa metin verileri üzerinde **ön-eğitimden (pre-training)** geçirilerek dilin yapısını, gramerini, anlamsal ilişkileri ve dünya hakkındaki temel bilgileri öğrenen bir modeldir.

Süreç:

Bu eğitim, "**denetimsiz (unsupervised)**" veya "**kendi kendine denetimli (self-supervised)**" bir süreçtir. Model, verideki istatistiksel kalıpları kendi kendine öğrenir.

Amaç:

Temel amaç, belirli bir soruya cevap vermek veya bir talimatı yerine getirmek değil, dili derinlemesine anlamak ve tutarlı metinler üretebilmektir. Bu modeller, işlenmemiş bir potansiyeldir.



Temel Modellerin İki Yüzü: Dili Nasıl Öğrenirler?

Causal Language Model (Causal LM)

- **Çalışma Prensipleri:** Bir dizideki bir sonraki kelimeyi (token) tahmin eder. ("Next Token Prediction")
- **Üretim Şekli:** Otoregresif (Autoregressive). Tahmin edilen her yeni kelime, bir sonraki kelimenin tahmini için girdinin bir parçası haline gelir. Bu, zincirleme metin üretimi sağlar.
- **Örnekler:** GPT serisi, Llama, Falcon.

Türkiye'nin başkenti []

Ankara'dır

Masked Language Model (Masked LM)

- **Çalışma Prensipleri:** Cümlelerin ortasındaki maskelenmiş bir kelimeyi, hem önceki hem de sonraki kelimeleri kullanarak (çift yönlü bağlam) tahmin eder.
- **Kullanım Alanı:** Metin sınıflandırma, anlamsal anlama gibi görevlerde bağlamı daha derinlemesine kavrar.
- **Örnekler:** BERT.

Güneş, Güneş Sistemi'nin [MASK] yer alan yıldızdır.

merkezinde

Potansiyelden Performansa: Talimat Anlayan Model (Instruct Model)

Problem

Foundation modeller, bir sohbet ortağı değildir. "Gökyüzü neden mavidir?" sorusuna, soruyu cevaplamak yerine cümlenin devamını getirecek anlamsız bir metinle karşılık verebilirler.

Çözüm: İnce Ayar (Fine-Tuning)

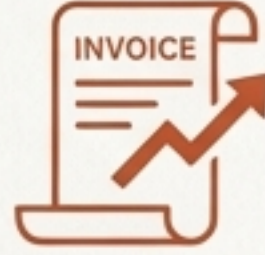
- **Süreç:** Modeli, yüksek kaliteli "talimat-cevap" çiftlerinden oluşan bir veri setiyle denetimli (supervised) olarak yeniden eğitime işlemidir.
- **Veri Seti:** "Aşağıdaki metni özetle: [Metin]" -> "Özet: [Özet]" gibi binlerce örnek içerir.
- **Sonuç:** Komutları anlayan, soruları yanıtlayan, özetleme yapabilen ve bir asistan gibi davranabilen bir **Instruct Model** ortaya çıkar.



Geleneksel Fine-Tuning: Güçlü Ama Pahalı

Zorluklar

Hesaplama Maliyeti: Çok sayıda güçlü GPU'ya ve günlerce süren eğitime ihtiyaç duyar.



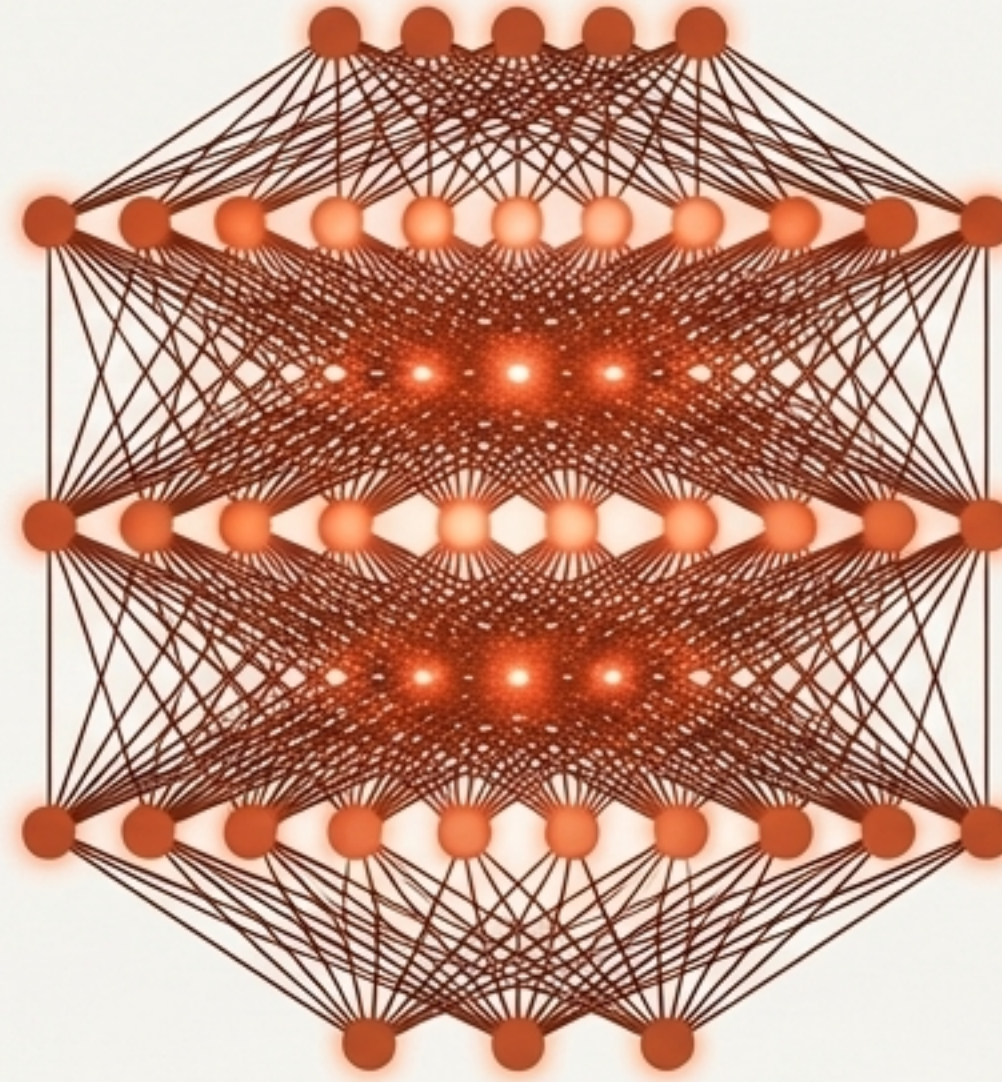
Bellek Gereksinimi:

Sadece modelin ağırlıkları bile (örn: 8 milyar parametre için ~32 GB VRAM) devasa bir bellek kaplar.



Depolama Sorunu:

Eğitilen her yeni model, orijinal modelin tam bir kopyasıdır. Bu da onlarca gigabaytlık yeni dosyalar anlamına gelir.



Yöntem: Full Fine-Tuning

Modelin sahip olduğu milyarlarca parametrenin **tamamının** eğitim verisiyle güncellenmesi işlemidir.

Sonuç

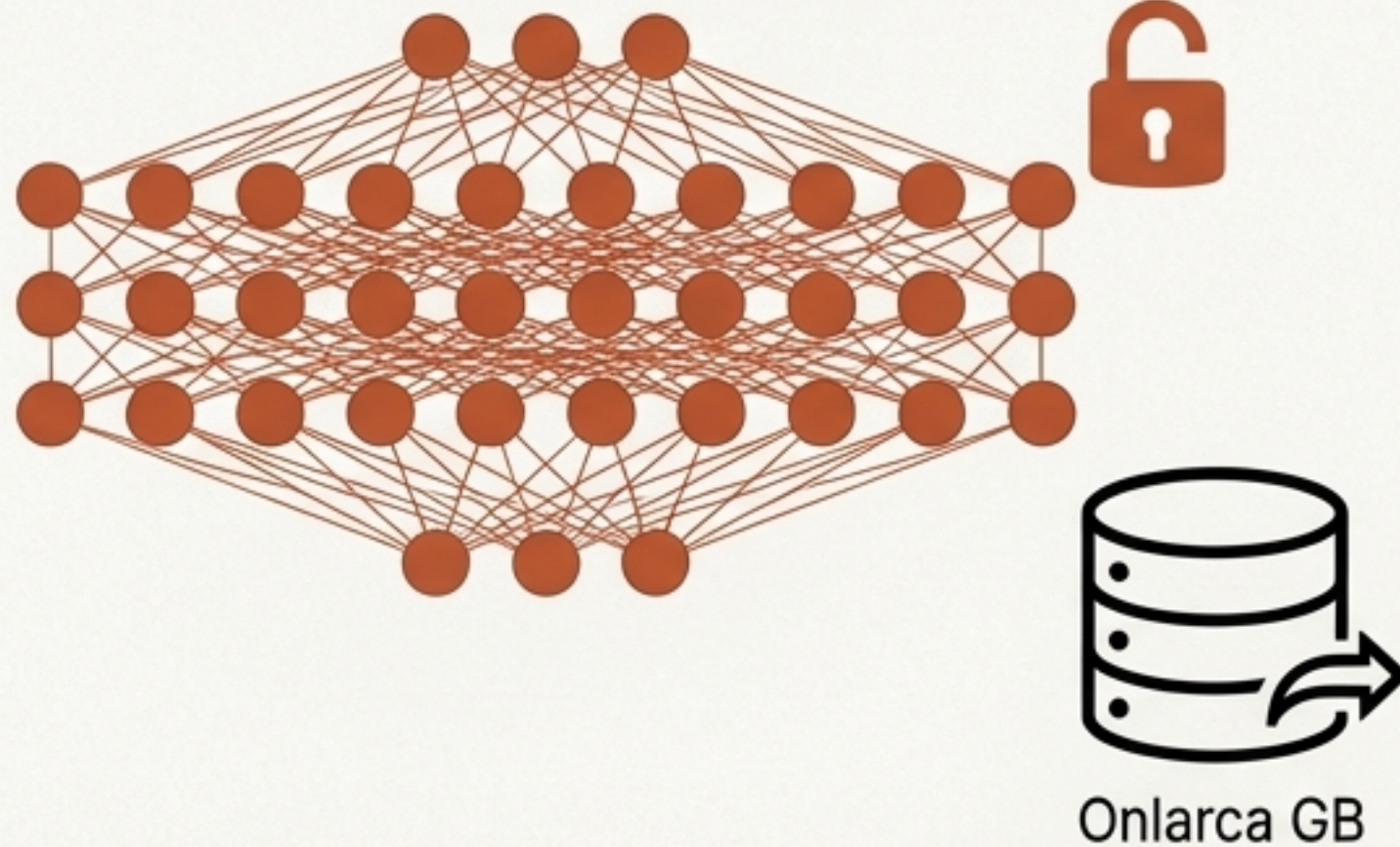
Bu yöntem, büyük şirketler ve araştırma laboratuvarları dışında çoğu geliştirici için pratik değildir.

Verimlilik Devrimi: PEFT ve LoRA

Çözüm: PEFT (Parameter-Efficient Fine-Tuning)

- **Temel Fikir:** Milyarlarca parametreye sahip orijinal modeli değiştirmek yerine **dondurun**. Sadece çok küçük sayıda **yeni** parametre eğitin.
- **Avantajları:** Hesaplama, bellek ve depolama maliyetlerini dramatik şekilde azaltır. Tüketici elektroniği donanımlarında bile fine-tuning yapmayı mümkün kılar.

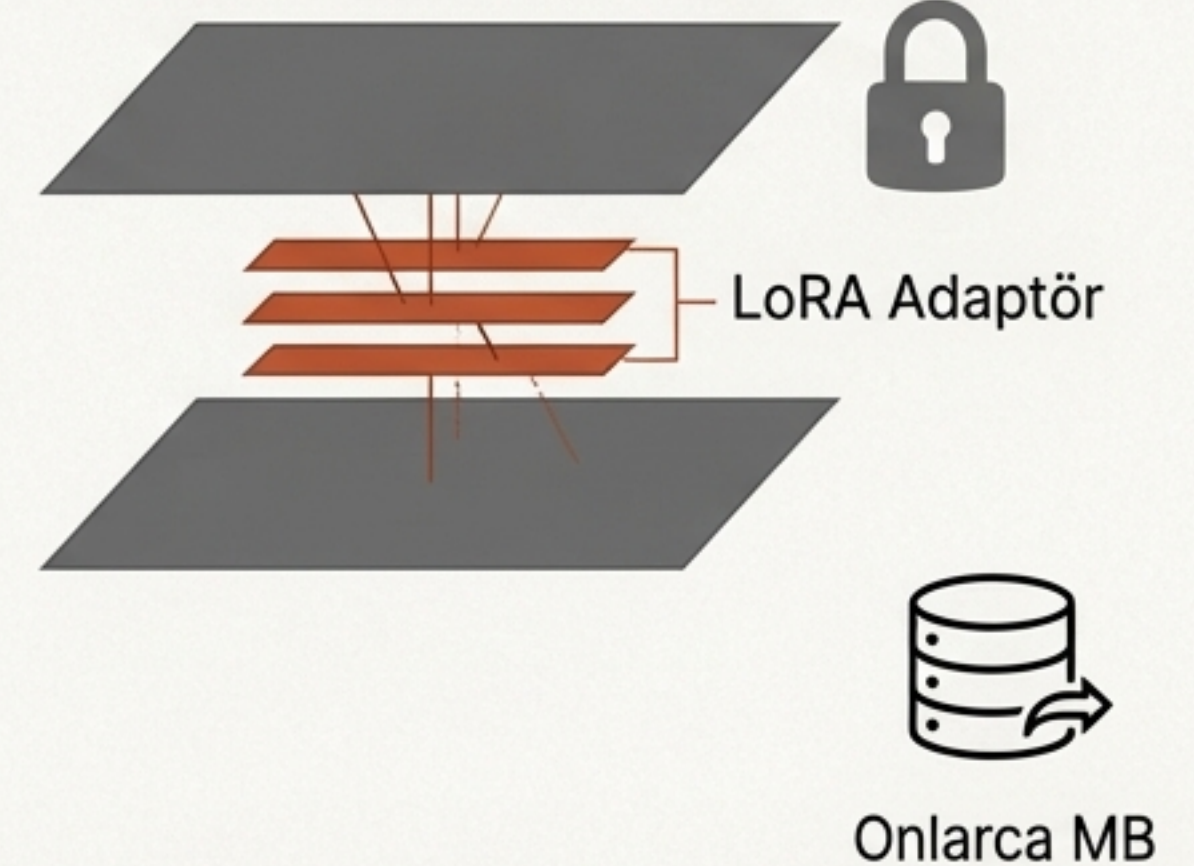
Full Fine-Tuning



En Popüler PEFT Yöntemi: LoRA (Low-Rank Adaptation)

- LoRA, bu "yeni" ve "küçük" parametreleri, "adaptör" adı verilen küçük matrisler olarak modelin mevcut katmanlarına entegre eder.
- Eğitim sonunda sadece bu küçük adaptörler (onlarca MB) saklanır. Orijinal devasa model (onlarca GB) ise sabit kalır.

LoRA (Low-Rank Adaptation)



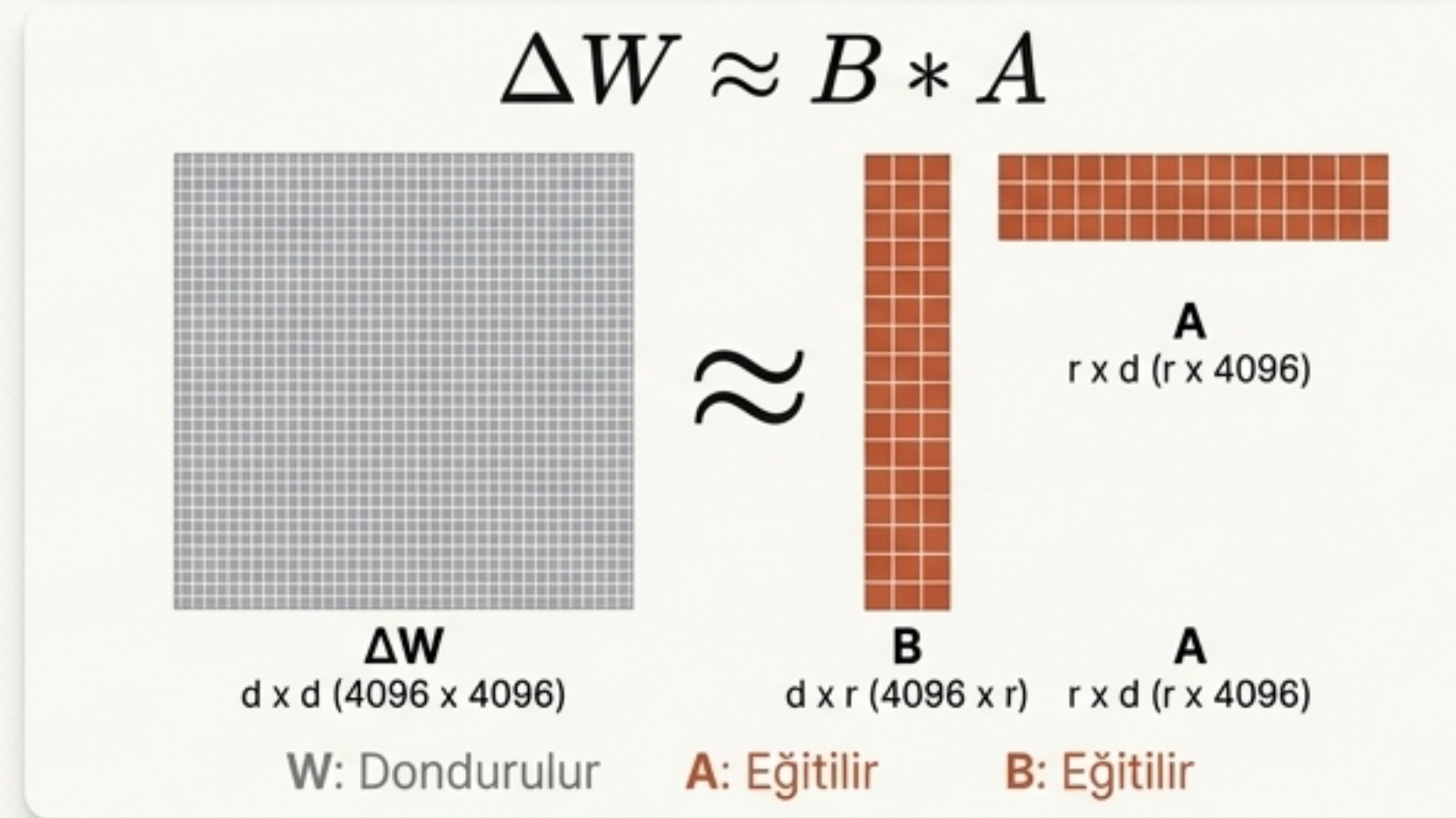
LoRA'nın Perde Arkası: Matrislerin Gücü

Problem

Büyük bir ağırlık matrisini (W , örn: 4096×4096) güncellemek, yani ΔW matrisini hesaplamak ve saklamak çok maliyetlidir. Bu matris 16.7 milyon parametre içerir.

LoRA'nın Sezgisel Çözümü

ΔW güncelleme matrisinin, kendisinden çok daha küçük iki "dar ve uzun" matrisin (B ve A) çarpımıyla yaklaşık olarak temsil edilebileceği varsayımına dayanır. Bu işleme "düşük ranklı ayrıştırma" (low-rank decomposition) denir.



Parametre Kazancı ($r=8$ için)

ΔW 'yi eğitmek: $4096 * 4096 = 16,777,216$ parametre.

B ve A 'yı eğitmek: $(4096 * 8) + (8 * 4096) = 65,536$ parametre.

~250 kat daha az eğitilebilir parametre!

Modeli Çalıştırmak İçin Sıkıştırma: Quantization

Quantization Nedir?

Modelin ağırlıklarını temsil eden sayıların hassasiyetini (precision) düşürme işlemidir.

- **Analoji:** 3.14159265 gibi hassas bir sayıyı, daha az yer kaplayan 3.14 olarak saklamaya benzer. Bir miktar hassasiyet kaybedilir ama depolama ve işlem hızı kazanılır.
- **Süreç:** Ağırlıklar genellikle 32-bit veya 16-bit kayan noktalı sayılarla (FP32/FP16) saklanır. Quantization ile bu sayılar 8-bit, 4-bit (INT8/INT4) gibi daha düşük bit
- **Sonuç:**erine dönüştürülür.
 - **Avantaj:** Daha küçük model boyutu (daha az VRAM/RAM), daha hızlı çıkarım (inference).
 - **Dezavantaj:** Performansta küçük bir düşüş yaşanabilir.

Yaygın Sıkıştırma Seviyeleri (TheBloke Notasyonu)

Q4_K_M, Q5_K_S, Q8_0 gibi isimlendirmeler, kullanılan bit sayısını ve sıkıştırma yönteminin detaylarını belirtir.



Sıkıştırma Seviyeleri ve Hassasiyet Kaybı

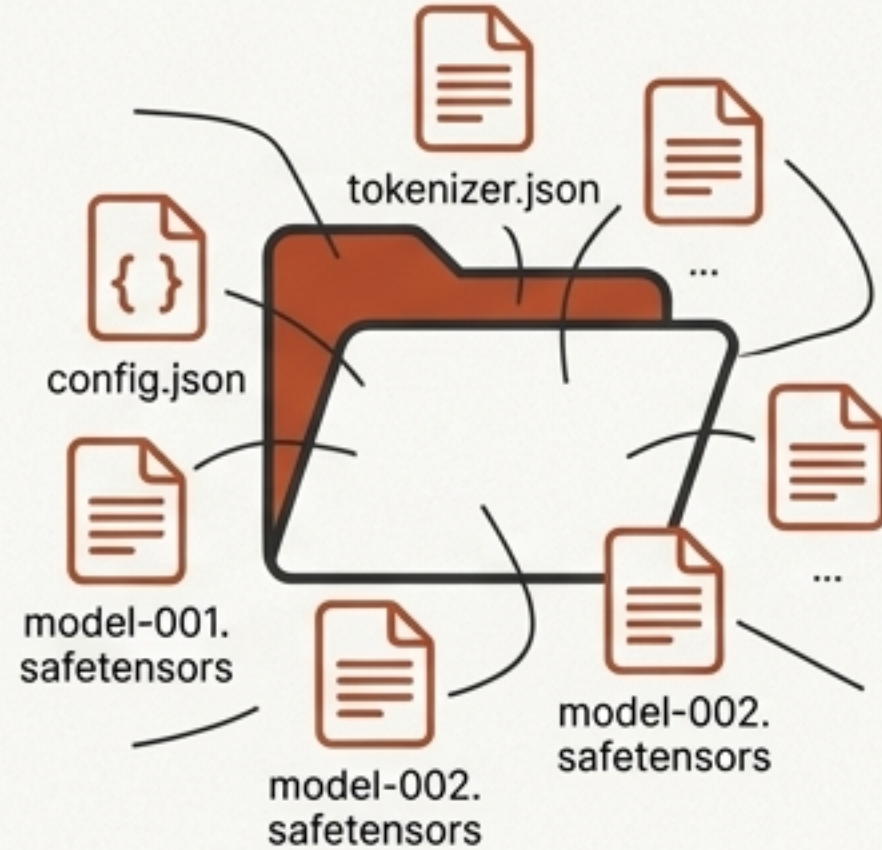
Ekosistemin Standart Paketleri: GGUF ve GPTQ

Problem

Eđitilmiş bir model, `config.json`, `tokenizer.json` ve onlarca ağırlık (`.safetensors`) dosyasından oluşan karmaşık bir klasör yapısına sahiptir. Bu, dağıtımı ve kullanımı zorlaştırır.

Çözüm

Her şeyi tek bir dosyada paketleyen standart formatlar.



GGUF (Georgi Gerganov Universal Format)

- Modelleri CPU'larda ve tüketici sınıfı GPU'larda çalıştırmak için fiili standarttır.
- Model mimarisini, (çoğunlukla quantize edilmiş) ağırlıkları ve tokenizer'ı tek, taşınabilir bir `.gguf` dosyası içinde birleştirir.
- **llama.cpp** ve **Ollama** gibi popüler araçlar bu formatı kullanır.



GPTQ

Özellikle GPU'lar için optimize edilmiş bir başka popüler quantization yöntemi ve formatıdır.

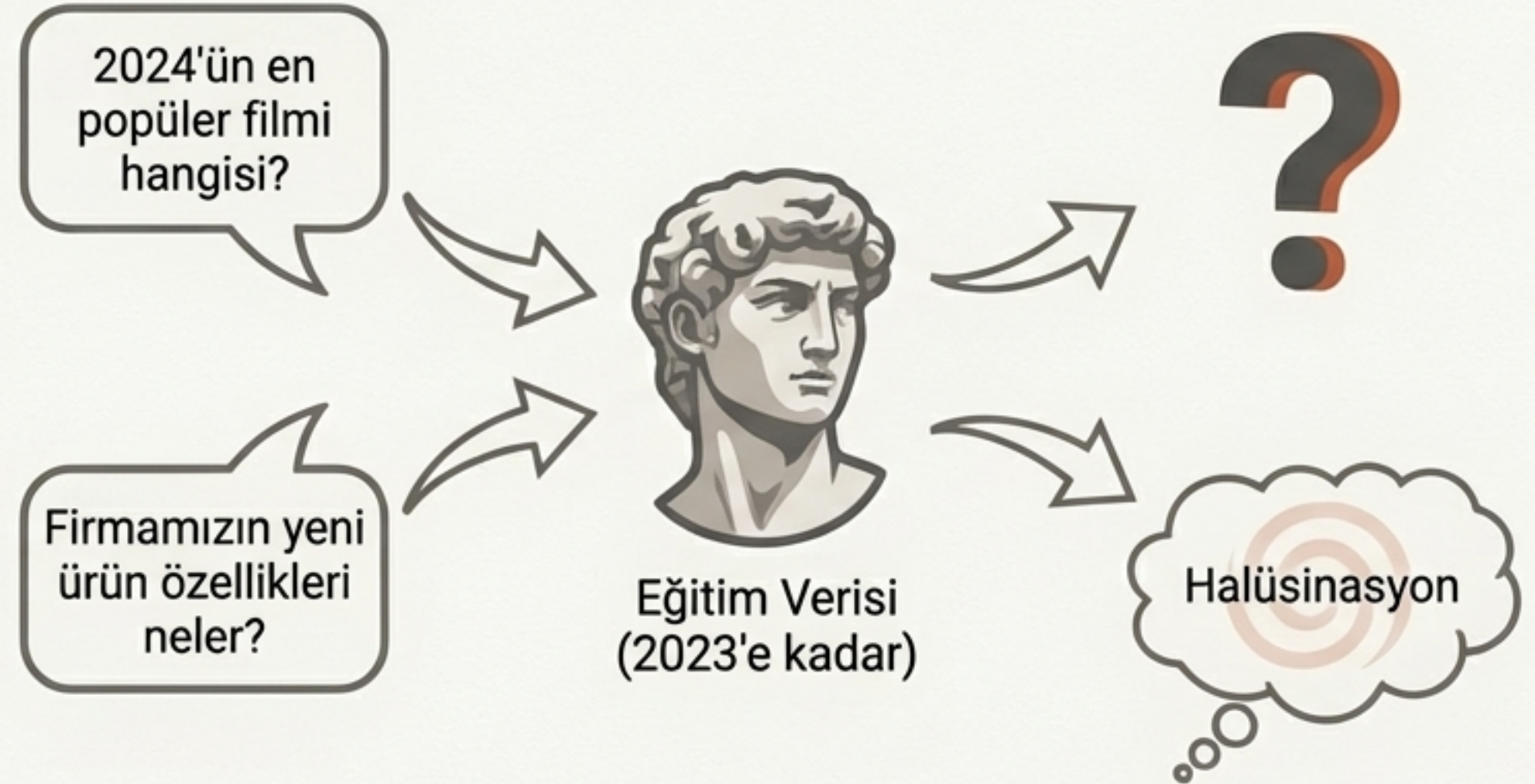
Zeki Ama Bilgisiz: Bilgi Boşluğu ve Halüsinasyon Sorunu

Temel Problem

Büyük dil modelleri, "bilgiye sahip" varlıklar değildir. Sadece eğitim verilerindeki istatistiksel kalıplara dayanarak dayanarak bir sonraki kelimeyi tahmin ederler. Bilgileri, eğitildikleri tarihle sınırlıdır ve sabittir.

Sonuçları:

1. **Eskimiş Bilgi:** Güncel olaylar hakkında cevap veremezler. ("Bugün borsa nasıl kapandı?")
2. **Özel Bilgi Eksikliği:** Şirketinizin iç dokümanlarını veya kişisel veritabanınızı bilmezler. ("3. çeyrek satış raporumuzu özetle.")
3. **Halüsinasyon:** Bilmedikleri bir konuda cevap vermeye zorlandıklarında, kulağa mantıklı gelen ancak tamamen yanlış olan bilgiler uydururlar.

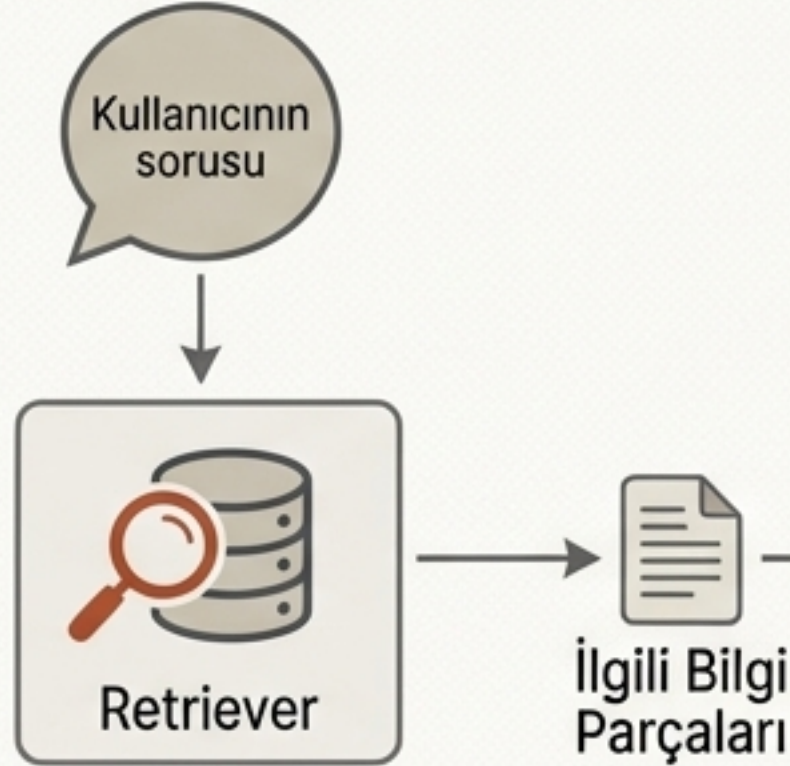


Modeli Gerçekliğe Bağlamak: RAG (Erişim Destekli Üretim)

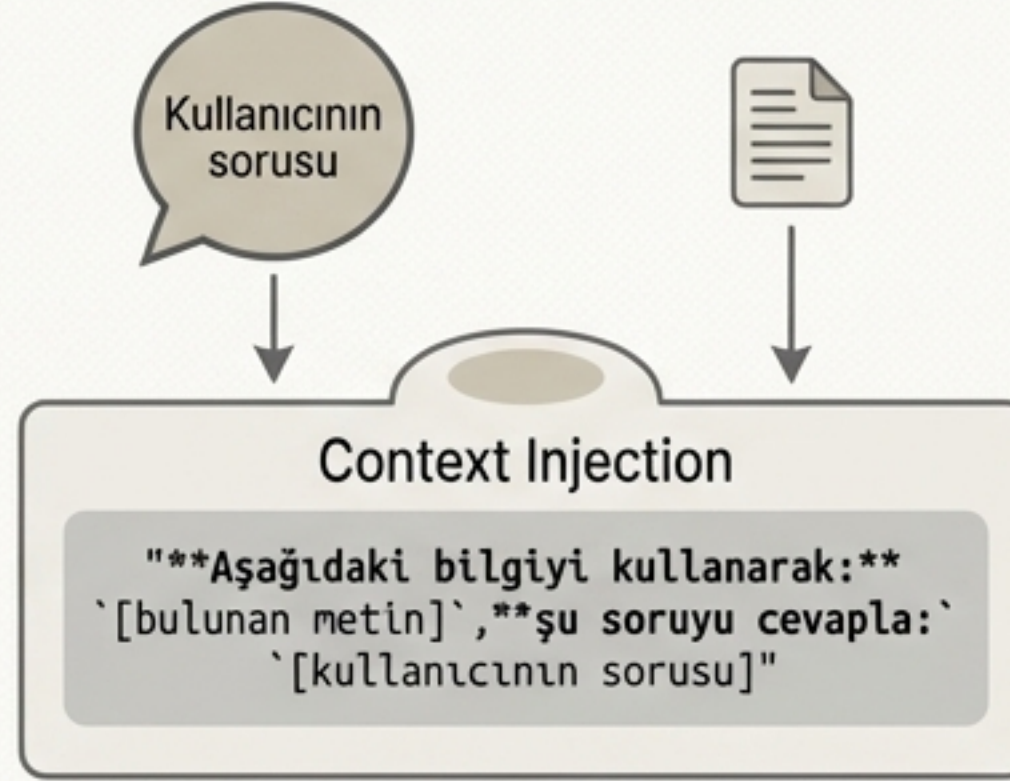
LLM'in, bir cevap üretmeden önce konuyla ilgili harici bir bilgi kaynağından veri alarak "temellendirilmesi" (grounding) tekniğidir. Modelin hafızasına değil, anlık olarak sunulan kanıta dayanmasını sağlar.

Basitleştirilmiş 3 Adımda RAG Akışı

1. ERİŞ (Retrieve)



2. DESTEKLE (Augment)



Kullanıcının sorusu geldiğinde, önce ilgili bilgi tabanında (dokümanlar, veritabanı, API) bir arama yapılır ve en alakalı bilgi parçaları bulunur.

Bulunan bu bilgiler, orijinal soruyla birleştirilerek LLM'e yeni bir "genişletilmiş" prompt olarak sunulur. Bu işleme "Context Injection" denir.

3. ÜRET (Generate)



LLM, sadece kendisine sunulan bu bağlama (context) dayanarak bir cevap üretir. Bu, halüsinasyonu neredeyse tamamen ortadan kaldırır.

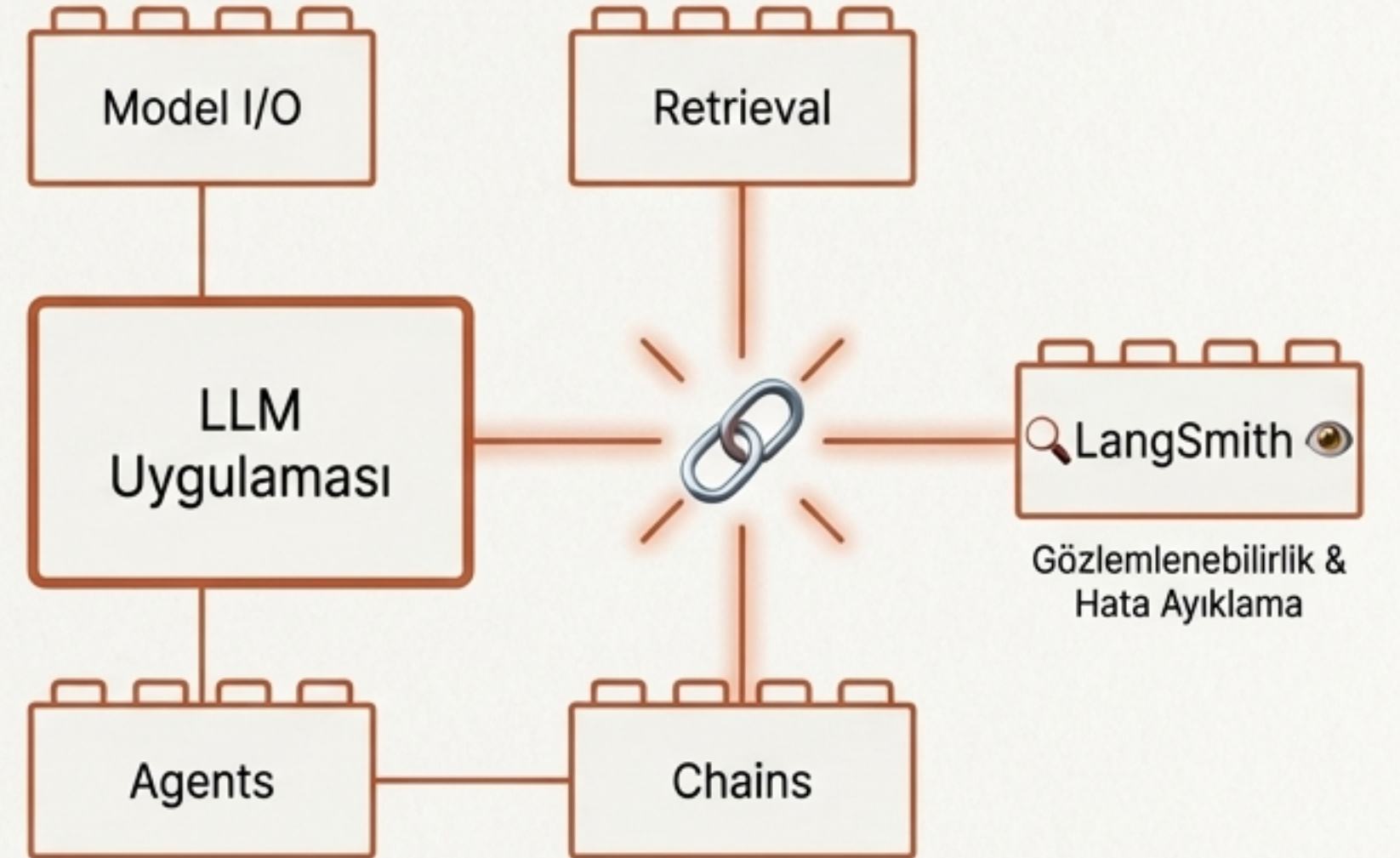
Fikirden Uygulamaya: Geliştirme Çerçevesi Olarak LangChain

LangChain Nedir?

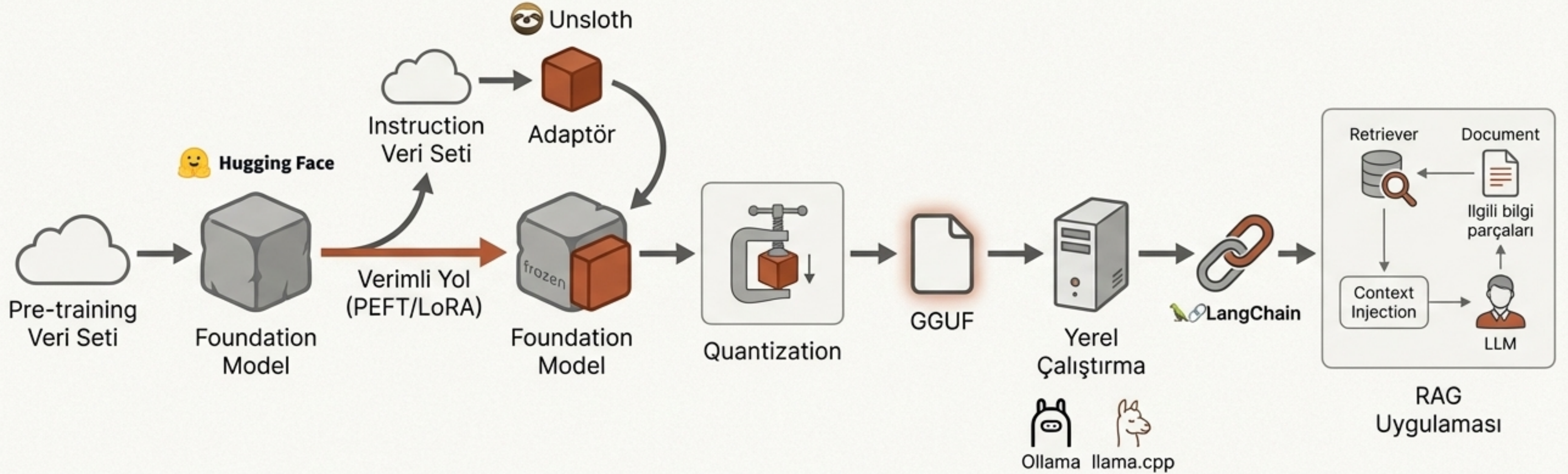
Dil modelleri tarafından desteklenen uygulamalar geliştirmek için bir **açık kaynaklı** çerçevedir. Karmaşık iş akışlarını modüler bileşenleri birbirine "zincirleyerek" (chaining) oluşturmayı kolaylaştırır.

Temel Yetenekleri

- **Ajanlar** (Agents): LLM'lerin, eylemde bulunmak için "araçları" (API'ler, veritabanı sorguları, web araması) kullanmasını sağlar.
- **Zincirler** (Chains): LLM çağrılarını RAG gibi diğer bileşenlerle birleştirmenin temel yoludur.
- **LangSmith**: Karmaşık LLM uygulamalarını izlemek, test etmek ve hatalarını ayıklamak için bir gözlemlenebilirlik (observability) platformudur. Ajanların "düşünce sürecini" adım adım gösterir.
- **LangGraph**: Doğrusal olmayan, döngüsel ve daha karmaşık ajan davranışları oluşturmak için kullanılır.



Ekosistem Haritası: Tüm Parçalar Bir Arada



Yolculuğun Özeti: Temel Çıkarımlar



Ham Potansiyel (Foundation Model): Dili anlar ama özel görevleri yerine getiremez. “Pre-training” ile doğar.



Uzmanlaşma (Fine-Tuning): Modeli belirli görevler için eğitir. **PEFT/LoRA** gibi verimli yöntemler, bu süreci herkes için erişilebilir kılar.



Verimlilik (Quantization & GGUF): Modelleri yerel cihazlarda bile çalışacak kadar küçültür ve hızlandırır.



Gerçeklik (RAG): Modelleri harici ve güncel bilgiyle “temellendirerek” halüsinasyonları önler ve güvenilir hale getirir.



Orkestrasyon (LangChain & Ollama): Tüm bu bileşenleri bir araya getirerek güçlü ve akıllı uygulamalar oluşturmayı sağlayan modern araçlardır.